算法一：

暴力模拟每次操作。

时间复杂度：$O(NQ)$。

期望得分：10-20 分。

算法二：

对于整棵树形成一条链的情况：

我们考虑用平衡树维护链上每个节点上次被采摘的时间，不难发现我们维护的是一段一段相等的时间区间，而对于每次采摘都至多让不同的时间区间增多 1。我们可以对于上次采摘时间相同的节点，用主席树 $O(logN)$计算这一段连续的节点产生的贡献，其中(容量 li/速度 vi)向上取整结果小于等于时间间隔的节点会被填满对答案有容量(li)的贡献，其余的节点会产生速度(vi)*时间间隔(△t)的贡献。以时间段为势能函数分析，每次操作最多势能加 1，而每用主席树计算一次势能-1。

时间复杂度：$O((N+Q)logn)$。

期望得分：40-50 分（结合算法一）。

算法三：

对于能量泉没有容量限制的情况：

对于每个能量泉，将(dfs 序位置，深度)作为二元组放到直角坐标系上，则每一次采摘的能量泉对应一个矩形内的所有点。考虑用 KD 树维护这些点，每次操作期望影响$\sqrt{N}$个单点与$\sqrt{N}$个 kd 树上的矩形，可以很容易通过打标记之类的方式维护答案。

时间复杂度：$O(NlogN+Q\sqrt{N})$。

期望得分：40-50 分（结合算法一）

算法四：

结合算法二和算法三的思想，每次操作的时候先找到这些单点和矩形，然后从每个矩形继续 dfs 到有标记的小矩形处，用与算法二类似的方式可以 $O(logN)$计算贡献，这里需要用到可持久化线段树或平衡树的合并。以当前有标记的矩形个数进行势能分析，每次操作期望增加$\sqrt{N}$的势能，每次计算一次贡献-1 势能。

时间复杂度：$O(NlogN+Q\sqrt{N}logN)$。

期望得分：100 分